

Profile

이름: 서예현

영문명: SEO YEHYUN

전화번호: 010-9780-6603

E-mail: 0812yeh@naver.com

현주소지: 대전광역시 중구

학력 및 교육

2017.03~2025.05

대전대학교 생명과학과 중퇴

2026.03~2026.10

DW 아카데미 AIoT 기반 드론 영상 관제시스템 개발 과정 수강(K-Digital Training)

About Me

도메인 융합

한결같은 흥미를 느낀 생명과학 지식과 새로 접한 AI-데이터 기술을 결합하여 실질적 문제 해결을 도모합니다.

지속 성장

천문학적 단위의 데이터를 분석, 정리하는 과정 자체를 즐기며, 새로운 기술을 빠르게 습득합니다.

개발자로서의 목표

단순 분석뿐만 아니라, 문제 해결에 유의미한 가치를 더하는 데이터 분석가 및 AI 개발자를 지향합니다.

개인 홈페이지

GitHub: <https://github.com/yehyun-42/yehyun-42.github.io>

Notion: https://app.notion.com/p/3a7bfca0450d8043b083da080a30e79a?source=copy_link

포트폴리오 목차

데이터 분석

개인 프로젝트 주제: 폭염과 산불이 오존 농도에 미치는 영향 분석

AI를 활용한 드론 관제 시스템

개인 프로젝트: AI 객체감지 (Keras, YOLOv5)

팀프로젝트: 유기동물 보호소 관제 시스템

데이터 분석

Tech Stack

Language	Python
IDE	Jupyter
Data & Viz	Pandas, NumPy, Matplotlib, Seaborn

개인 프로젝트 주제: 폭염과 산불이 오존 농도에 미치는 영향 분석

사용 자료: **Western North American FLEXPART Back Trajectory 1994-2021 Merge Data**
(NASA 제공)

수립한 가설

- 높은 수치의 오존 농도가 발생하였던 시기에는 폭염 및 산불 발생 시기가 일치할 것.
- 폭염의 경우 지구 온난화의 영향을 받으므로, 지구의 기온이 높아진 현재에 더 높은 오존 농도가 관측될 것.

가설 검증에 필요한 데이터

- 연, 월, 일별 기준치 이상 오존 수치.
- 고농도 오존 수치 감지 횟수가 가장 많은 월별, 일별 폭염 및 산불 기록.

데이터 전처리 및 분석 기준 설정

- 분석 기준치 설정

기준 수치: **70ppbv** (미국 오존주의보 발령 기준)

기준 설정 목적: 분석 대상이 '고농도 오존'이므로 해당 기준을 도입하여 필요한 데이터를 별도의 피벗테이블로 분리하여 분석 진행.

- 이상치 처리 방침

처리 방법: **이상치 미제거**

방침 설정 목적: 폭염 및 산불로 인해 관측되는 고농도 오존 수치를 분석 대상으로 지정하였으므로 이상치로 분류될 수 있는 수치들 또한 분석 대상으로 지정.

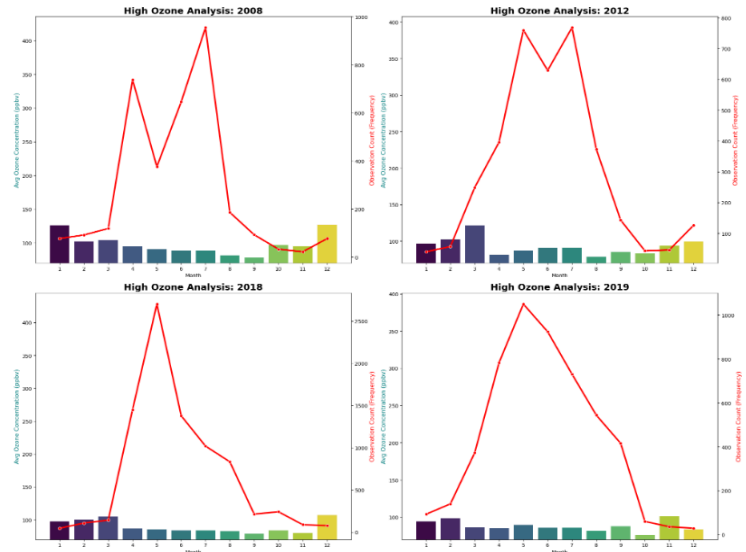
가설 검증

가설 1. 높은 수치의 오존 농도가 발생하였던 시기에는 폭염 및 산불 발생 시기가 일치할 것.

고농도 오존 수치 관측 횟수가 가장 많은 해(2008, 2012, 2018, 2019년)의 월별 수치를 시각화.

- 여름(6~8월)의 고농도 오존 관측 빈도수 급증.

- 7~8월의 경우 폭염이 원인인 것으로 예측. 6월, 9월을 기준으로 관측된 농도가 가장 높은 월을 선정. 해당 월의 폭염 및 산불 기록 검색.

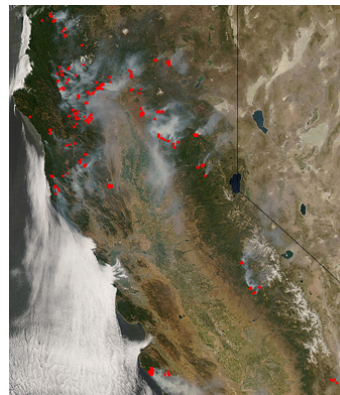


2008년의 경우, 캘리포니아 지역에서 기록적인 폭염 발생. 폭염 발생 2달 전 앨버타 주에서 대규모 산불 발생.

2018년도 캘리포니아에서 위성으로 관측되는 규모의 화재 발생.

산불 및 폭염과 오존 농도의 직접적인 상관 관계를 입증.

관측 횟수가 가장 많은 해의 월별 고농도 오존 평균 수치 (막대그래프), 관측 수치(선그래프)
y축: 오존 농도 평균(막대), 오존 농도 관측 횟수(선)



NASA MODIS에서 발표한 캘리포니아 화재를 위성이 관측한 사진

The New York Times
<https://www.nytimes.com/2008/06/09/0008-HEAT/index-9>
Heat Wave Bakes East Coast - Slide Show
2008. 6. 9. — The National Weather Service issued heat advisories as temperatures were expected to exceed 100 degrees in many areas.
Fire Fighting in Canada
<https://www.firefightingcanada.com/fire-erupt-in-ca>
Fires erupt in several Alberta locations
2008. 5. 16. — May 16, 2008, Bruderheim, Alta. - Firefighters across a wide swath of north-central Alberta were abruptly pressed into service Thursday when ...

2008년 폭염 및 화재 뉴스

가설 검증

가설2. 폭염의 경우 지구 온난화의 영향을 받으므로, 지구의 기온이 높아진 현재에 더 높은 오존 농도가 관측될 것.

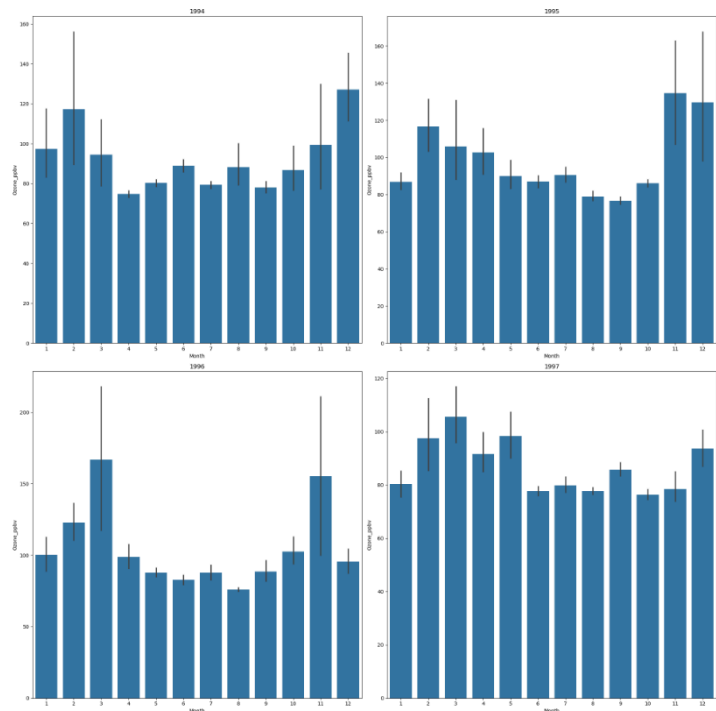
과거와 현재의 오존 농도 비교를 위해 기준치 이상의 오존 농도 관측 횟수가 가장 적은 년도(1994년~1997년)의 월별 고농도 오존 평균치 확인.

- 평균치는 대체로 100ppbv로 예상했던 대로 현재에 비해 수치가 낮음.
- 그러나 관측 횟수가 크게 증가하는 경우가 존재.

미국의 산불 통계 사이트 National Interagency Fire Center를 이용하여 데이터를 추출 한 연도별 산불 발생 통계 확인.

- 과거에도 산불의 발생 횟수는 현재와 유사.
- 피해 규모는 현재에 비해 현저히 적음.
- 피해 규모에 따라 대기오염물질 생성량 및 산불로 인한 열기가 영향을 미치는 영역이 증가함.

산불 발발 횟수 뿐 아닌 산불의 규모 또한 오존 농도에 영향을 미침.
또한 미국은 1990년 대기오염방지법 개정안이 발표됨.



1994년~1997년 월별 고농도 오존 수치 평균
y축: 오존 평균 농도, x축: 월

연도	산불 발발 횟수	피해 면적(에이커)
2019	50,477	4,664,364
2018	58,083	8,767,492
2012	67,774	9,326,238
2008	78,979	5,292,468
1997	66,196	2,856,959
1996	96,363	6,065,998
1995	82,234	1,840,546
1994	79,107	4,073,579

데이터 분석 결과 정리

산불 및 폭염과 오존 농도의 직접적 상관 관계 확인.

- 6~8월 여름철 및 산불 발생 시기와 다수의 고농도 오존 관측 시기가 일치함을 확인함.

과거에 비하여 높은 오존 수치 확인.

- 과거와 현재를 비교하였을 때 오존 농도가 현저히 높아졌음을 확인함.
- 산불의 발발 횟수는 과거와 변화가 없거나, 과거가 더 잦음을 확인함.

결론

1. 자연재해 조기 대응

산불의 발발 횟수 뿐 아닌 산불의 규모 또한 오존 발생에 영향을 미침.

산불의 조기 진압 등을 통한 방지 필요.

질소산화물 저감 장치 등의 대기오염물질 처리 기기의 적극적 도입 필요.

2. 법적 규제 및 사회적 노력 필요

1990년 대기오염방지법 개정 직후인 1990년대 중반(1994년도)에 오존 농도가 낮은 수치를 기록함.

이를 통해 시민이나 기업의 노력으로 오존 농도를 낮출 수 있음을 알 수 있음.

프로젝트 회고 및 개선사항

1. 도메인 지식 필요성

- 분석 시작 직후 시각화를 중요시 하여 미국 오존주의보 발령 기준을 오인. 이후 재확인을 거쳐 70ppbv의 기준을 재정립.
- 분석 신뢰도를 위해 전후 도메인 조사가 필수적임을 배움.

2. 가설 수립 과정의 체계화

- 분석 목표를 잃지 않도록 가설을 구체화하여 문서로 작성, 관리하는 습관을 형성.
- 명확한 문제 정의가 곧 명확하고 필요한 데이터 분석으로 이어짐을 배움.

3. 도구 활용 능력

- Python 및 Seaborn의 세부적 기능 파악.
- 구상한 분석 및 시각화를 위하여 사용하는 도구의 구체적인 기능에 의거한 계획 수립이 가능해짐.

AI를 활용한 드론 관제 시스템

Tech Stack

Language	Python
IDE	VS Code
Database	SQLAlchemy, Flask-Migrate
Object Detection	YOLOv5, Keras
Web streaming server	Flask
Hardware	ESP32-CAM

개인 프로젝트: AI 객체감지 (Keras, YOLOv5)

수행 목적: Keras 기반 모델과 실시간 객체 탐지에 특화되어 있는 YOLOv5 모델의 특징 비교 및 실습

1. Keras를 이용한 객체감지

Keras 감지를 위해 학습한 오브젝트

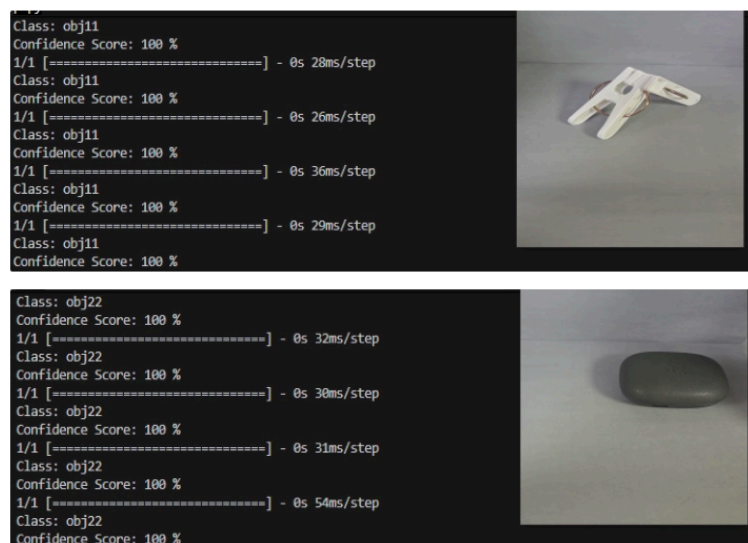
- obj11: 빨래집게
- obj22: 무선 이어폰

라벨링: Labelimg 이용

학습: Teachable Machine 이용

학습 결과

- 테스트 결과 각 객체에 대한
예측 신뢰도가 100%로
출력됨.



학습 검증

2. YOLOv5를 이용한 객체감지

YOLOv5 실시간 객체 감지를 위해 학습한 오브젝트

- 정지, 진입금지, 횡단보도, 속도제한, 천천히 등의 표지판 이미지
- 사람, 차, 오토바이 이미지

라벨링: Labelimg 이용

학습: Colab 이용

시행착오

1차 시도

- 낮은 예측 신뢰도(80% 미만).
- 표지판 이미지의 인식 혼동.

파악한 원인: 화질 저하에 따른 가독성 저하

개선사항: 이미지 캡처 시의 화질을 고려하여 객체별 촬영 시간을 증가.

각 객체 별 조건을 설정하여 라벨링 진행.

2차 시도

- 예측 신뢰도의 수치가 80%~90%로 증가.
- 이미지간의 인식 혼동.

파악한 원인: 1차 시도에 비하여 부족한 데이터 양.

개선사항: 1차 시도에서 사용한 데이터와 통합하여 학습.

3차 시도

- 70%~80%로 낮아진 예측 신뢰도.
- 인지하지 못하는 객체가 늘어나고, 인식 혼동 문제 또한 여전함.

파악한 원인: 학습용 이미지 자체적인 해상도 저하.

개선사항: 224, 284의 크기로 진행하던 학습용 이미지 캡처를 640, 680으로 조정.

최종 학습 결과

- 대부분의 예측 신뢰율이 90% 이상 출력됨.
- 횡단보도, 정지 표지판이 함께 있는 경우 인식이 잘 되지 않음.
- 스튜디오 외의 환경이나 일정 거리 이상 멀어졌을 경우 인식 되지 않음.



최종 학습검증

팀 프로젝트 주제: 유기동물 보호소 관제 시스템

문제 인식

사설 유기견 보호소 봉사활동 중, 극소수의 인원이 대규모의 동물을 관리하며 겪는 인력 부족을 체감. 이러한 인력 부족으로 인한 다양한 문제에 대해 기술을 통하여 해결할 수 있는 방안을 찾기 시작함.

기술적 동기 부여

국립과천과학관에서 실제 응용 중인 딥러닝 기반 객체 감지 기술을 관람.

강의실에서 배운 기술을 실제 현장에 적용하여 사람의 업무 부담을 덜어내는 시스템의 구축을 목표함.

주제 선정 이유

- 유기동물 보호소라는 소수의 인원이 대규모의 동물을 관리하는 환경에서 인력 부족으로 인한 돌발 및 위기 상황 대처 방안 수립
- 드론과 AI를 이용하여 사람의 업무 강도를 낮추고 보다 효율적인 관리를 지향.

벤치마킹 레퍼런스

- 세종 과학 기지 월동연구대의 펭귄 개체수 파악 기술
드론을 이용하여 펭귄의 서식지를 촬영, 해당 영상과 AI를 이용하여 펭귄의 개체수 및 동지를 파악하는 기술. 보호 동물 개체수 감지 기능 구축에 참고.
- 경기도 양평군, 강원도 평창군 등에 설치 된 로드킬 방지 스마트 시스템
국도 상의 야생동물 로드킬을 방지하는 시스템. 인공지능 기술 기반의 CCTV 및 레이더 센서를 설치하여 야생 동물 출현을 감지. 야생동물 감지 기능 구축에 참고.
- 홈캠의 실시간 모바일 푸시 알림
지정된 범위를 촬영하며 조건을 만족할 경우 사용자의 스마트폰 등으로 알람을 보내는 홈캠의 기능. 실시간 알림 시스템 구축에 참고.

프로젝트 내 담당 역할: 백엔드 개발

- Database & Data Management
Database 작성 및 시각화
감지내역 Database 연동 및 관리
- Streaming Troubleshooting Fix
ESP32-CAM 송출 오류 해결
- 경보 알림 기능
메일 경보 발송: E-mail 연동

1. Database & Data Management

초기 구상 테이블

사용자	Users
개체수 감지 내역	Animals
야생동물 감지 내역	Dangers
경보 발송	Alarms

특이사항

- 경보 발생 DB의 경우 타 DB 내역 참조 필요.
야생동물(이상객체) 감지 내역 DB 내의 고유
번호, 감지 일시 참조.
- 사용자 DB 내의 E-mail 참조.

최종 확정 테이블

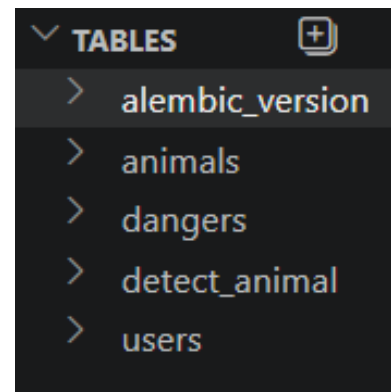
사용자	Users
보호 중인 개체수	Animals
감지 할 야생동물	Dangers
감지 내역	Detect_animal

테이블 수정 이유

- 보호 중인 유기 동물의 개체수(기준치)의 필요성 파악.
 - 효율성을 고려하여 경보 발송 테이블을 별도로 정의, 관리 하는
것이 아닌 감지 내역을 즉각적으로 판단하여 알람 발송 여부를
검토하도록 수정.
 - 개체수 감지의 경우 특정 조건(개체수 미달) 만족 시 알람 발송.
경보 발생 여부의 시각적 확인을 위해 'detect_alarm' 컬럼을
boolean 값으로 추가.
- 시각화 대시보드 내에서 '미발송', '발송 완료' 상태로 확인 가능.

테이블명	시스템명	작성일자	2026-05-19
주제영역명	Stray Animal Protection System	작성일자	2026-05-19
테이블ID	users	테이블명	
테이블설명	사용자 DB		
번호	컬럼명	속성명	도메인
1	user_id	사용자 고유번호	N/A INT
2	user_name	사용자 이름	N/A VARCHAR(50)
3	user_email	사용자 이메일	N/A VARCHAR(255)
4	user_role	사용자 권한	N/A VARCHAR(50)
테이블명	시스템명	작성일자	2026-05-19
주제영역명	Stray Animal Protection System	작성일자	2026-05-19
테이블ID	animals	테이블명	
테이블설명	유기동물 DB		
번호	컬럼명	속성명	도메인
1	animal_id	유기동물 고유번호	N/A INT
2	animal_spec	유기동물 종류	N/A VARCHAR(200)
3	animal_count	유기동물 감지 마리수	N/A INT
4	animal_detect	유기동물 감지 일시	N/A DATETIME
테이블명	시스템명	작성일자	2026-05-19
주제영역명	Stray Animal Protection System	작성일자	2026-05-19
테이블ID	dangers	테이블명	
테이블설명	이상 객체 DB		
번호	컬럼명	속성명	도메인
1	danger_id	이상 객체 고유번호	N/A INT
2	danger_spec	이상 객체 종류	N/A VARCHAR(50)
3	danger_detect	이상 객체 감지 일시	N/A DATETIME
테이블명	시스템명	작성일자	2026-05-19
주제영역명	Stray Animal Protection System	작성일자	2026-05-19
테이블ID	alarms	테이블명	
테이블설명	알람 DB		
번호	컬럼명	속성명	도메인
1	alarm_id	알람 고유번호	N/A INT
2	danger_id	이상 객체 고유번호 참조	N/A INT
3	danger_detect	이상 객체 감지 일시 참조	N/A DATETIME
4	alarm_email	알람 발송 메일 대상	N/A VARCHAR(500)

테이블 정의서



최종 확정 된 DB

발송 완료

미발송

발송 완료

불린값의 시각화

순환 참조 방지

DB 구성 초기 단계의 구조: app.py 파일 내에 DB를 import.

관제 시스템의 규모가 증가함에 따라 순환 참조 및 유지보수 문제가 발생할 가능성 파악.

해결: extensions.py 파일을 생성하여 DB만을 담당하도록 설정. 이후 app.py에서 DB가 아닌 extensions.py를 import.

```
extensions.py X
project_SSA > apps > extensions.py > ...
1 from flask_sqlalchemy import SQLAlchemy
2
3 db=SQLAlchemy()
```

순환 참조 방지를 위한 extensions.py

DB 시각화 대시보드

DB 접근 및 관리 용이성을 위해 DB를 시각화.

DB 저장 내역을 사용자가 직관적으로 파악 가능하도록 데이터 필터링 및 가공 과정을 거쳐 시각화.

Animals 테이블의 경우 시각화 대시보드 내에서 즉각적인 수정이 가능하도록 연동.

- 보호 동물(key)과 개체수(value)를 딕셔너리로 지정.
- 사용자가 입력한 내용이 딕셔너리에 적용되도록 연동.

ID	감지된 동물	감지된 마리수	감지 일시	알람 발송 여부
#691	개(미달)	2마리	2026-06-18 17:01:52	발송 완료
#692	고양이(미달)	0마리	2026-06-18 17:01:52	발송 완료
#693	개:2마리	2마리	2026-06-18 17:01:52	발송 완료

ID	출몰한 야생동물 종류	출몰 일시
#155	외계인, 상어	2026-06-18 16:02:00
#154	용	2026-06-18 16:02:00

감지된 내역이 한글로 출력되는 DB 시각화 대시보드

각 테이블 연동 시 발생한 문제

- YOLO 감지내역과 Detect_animal 테이블, 기준치 딕셔너리와 Animals 테이블의 연동 불가.
원인: Tensor 값과 딕셔너리 값이 DB 테이블이 받을 수 있는 값으로 변환하지 않음.
해결: Tensor 값과 딕셔너리 컬럼 값을 직접적으로 추출, SQLite에 직접적으로 입력.

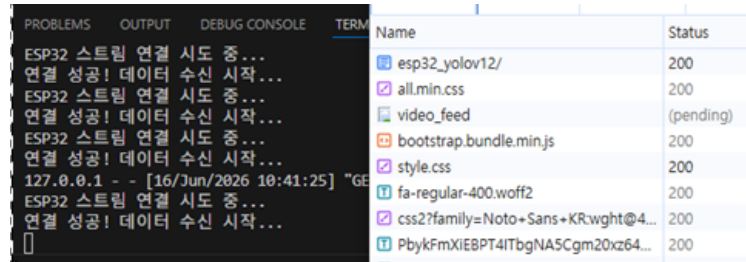
2. Streaming Troubleshooting Fix

ESP32-CAM 연결 문제

ESP32-CAM 자체 송신은 성공적으로 이루어짐.

Flask server에서 ESP32-CAM 영상을 수신하지 못함.

개발자 도구 내의 Network 탭을
확인해본 결과 ESP32-CAM 영상을
수신하는 Video_feed에 pending이
출력.



Name	Status
esp32_yolov12/	200
all.min.css	200
video_feed	(pending)
bootstrap.bundle.min.js	200
style.css	200
fa-regular-400.woff2	200
css2?family=Noto+Sans+KR:wght@4...	200
PbykFmXiE8PT4ITbgNA5Cgm20xz64...	200

개발 당시 발생하였던 ESP 송신 오류 중 하나.
ESP32 연결은 성공적이거나, Flask 서버에는 pending 상태가 지속되었다.

1차 시도: requests 방식

OpenCV 대신 requests 방식 적용.

- JPEG segment 문제 발생.
- 화면이 나오지 않거나, 깨지는 현상 반복.

2차 시도: Flask 비경유 방식

Flask를 경유하는 것이 아닌 ESP32-CAM 화면을 직접 송출

- 영상 송출은 성공적.
- YOLO를 이용한 감지 불가.
- 프로젝트의 목적이 YOLO를 이용한 실시간 객체 감지이기 때문에 해당 방법을 폐기.

3차 시도: 멀티 스레딩

- 하나의 함수에서 영상 수신, YOLO 감지, 영상 송출을 담당.
- 함수를 두 개로 나눔.

첫번째 함수: 영상 수신, YOLO 감지. 감지 된 프레임을 변수 current_frame에 저장.

두번째 함수: current_frame에 저장된 프레임을 Flask 화면에 송출.

- ESP32-CAM 영상 정상 송출 및 YOLO 감지 작동.

3. 경보 알림 기능

메일 경보 알림: E-mail 연동

메일 경보 발송 모듈과 DB 연동 구현.

YOLO 감지 내역과 DB를 연동시킨 것과 동일하게 SQLite를 직접적으로 호출.

조건에 맞는 E-mail을 참조

- filter_by를 통해 메일 수신 권한을 가진 사용자의 E-mail을 참조.

스레딩 방식을 이용하여 비동기 처리.

☆ [경보] 개체수 미달 감지 알림 

보낸사람 dhkang8817@gmail.com

2026년 6월 18일 (목) 오후 6:22

관리자님,

현재 개체수 미달이 감지되었습니다. dog 시스템을 확인해 주세요.

실제 발송된 경보 알림

```
email_thread = threading.Thread(target=send_alert_email, args=(animal,), daemon=True)
email_thread.start()

discord_thread = threading.Thread(target=send_discord_webhook, args=(type_string,), daemon=True)
discord_thread.start()
```

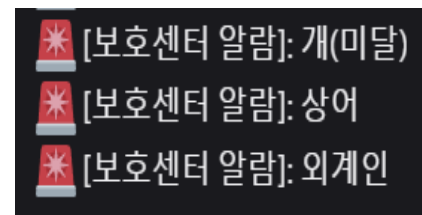
경보 발송 로직. 스레딩을 이용해 비동기처리 한다.

디스코드 경보 알림: Webhook 연동

메일 알림의 단점: 사용자가 메일에 접속해야만 확인 가능.

- 야생동물 출현 및 개체수 미달 상태 등의 문제는 즉각적 대처가 중요.
- 시각적, 청각적 접근성의 중요도 파악.
- 몇 가지 메신저 어플리케이션 중 개발 자유도가 높은 디스코드 웹후크를 선정.

메일 알림과 마찬가지로 스레딩 방식을 이용하여 비동기 처리.

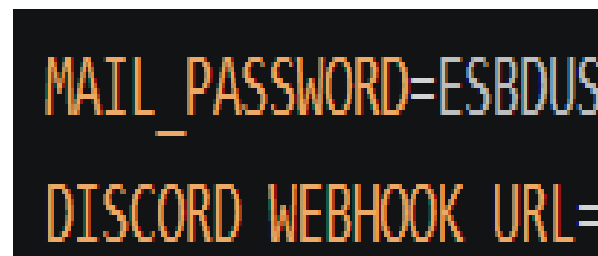


[보호센터 알람]: 개(미달)
[보호센터 알람]: 상어
[보호센터 알람]: 외계인

실제 발송된 디스코드 경보

보안성 증대

Mail_password 및 디스코드 웹후크 URL을 .env 파일로 분리하여 보안 강화.



MAIL_PASSWORD=ESBDUS
DISCORD_WEBHOOK_URL=

.env 파일로 비밀번호와 url을 분리하여 보안성을 강화하였다.

팀프로젝트 회고

반성 및 개선사항

1. 개체수 감지 시스템

실제 보호소에서 보호하는 동물은 수십, 수백 단위.

현 감지 로직: 미달 상태 최초 감지 후 10초간 유지될 경우에만 DB에 저장, 경보 발생.

실제 보호소 환경에서 적용되기 어려움. 보다 확실한 객체 인식 수단 필요.

부분 라벨링 등의 방법을 통해 객체 겹침을 해소할 수 있음.

2. 이상객체(야생동물) 감지 시스템

현 감지 로직: 이미 등록되어 있는 야생동물에 한하여 감지.

야생동물은 다양한 원인으로 서식지 변경이 일어남.

해당 지역에서 새로운 야생동물이 출몰할 경우, 학습 및 등록 과정을 거쳐야 하는 번거로움 존재.

벤치마킹 기술인 로드킬 방지 스마트 시스템의 로직: 움직이는 물체가 인식될 경우 해당 물체가 야생동물인지 판단.

해당 로직을 이용하여 '등록된 야생동물'이 아닌 '야생동물' 자체를 감지하도록 로직 변경 필요.

3. 지도 서비스

현 시스템: 야생동물이 출몰하면 경보 발생.

근본적인 해결 방안 수립을 위해 야생동물 출몰 위치가 기록되는 지도 제공.

야생동물 주요 출몰 지역 특정 및 야생동물 대처법 공유 수단 수립 필요.

성장

DB 구축 및 연동 경험

- ESP32-CAM으로부터 Flask 서버, YOLO, DB, 사용자의 경로를 이어나가는 파이프라인을 설계해봄.
- 필요에 맞춰 DB 테이블을 설계하고 구축.
- 서버 내부적으로는 YOLO가 감지한 내역을 DB에 저장하여 시각화하는 과정을, 서버 외부적으로는 사용자가 입력한 정보값을 DB에 저장하여 기준치와 연동하는 과정을 경험.

트러블슈팅 해결 및 방지

- 예상하지 못한 소프트웨어적 트러블슈팅을 경험하고, 해결해나감.
- 이후 발생 가능성이 있는 오류를 방지하고, 보안성을 강화함.

팀원간의 소통

- 프로젝트 진행 중 발생한 문제에 대해 홀로 고민하는 것이 아닌 팀원 전체와의 소통을 통해 방향을 정하고 해결해나감.